■ XSS Labs

PortSwigger Web Security Academy

Complete Writeup • All 9 Apprentice Labs • Full Solutions & Payloads

■ 9 Labs Solved	■ Apprentice Level	■ 2026

Category	Details
Lab Platform	PortSwigger Web Security Academy
Lab Type	Cross-Site Scripting (XSS)
Difficulty	Apprentice (All 9 Labs)
Topics Covered	Reflected · Stored · DOM-Based XSS
Tools Used	Burp Suite · Browser DevTools
Author	Piyush Kumar

■■ *This document is intended strictly for educational purposes and authorized security research. All labs are performed in an isolated, legal lab environment provided by PortSwigger. Never test techniques on systems you do not own or have explicit written permission to test.*

Table of Contents

LAB 01	REFLECTED XSS	■ SOLVED	APPRENTICE

Reflected XSS into HTML Context with Nothing Encoded

■ DESCRIPTION

This lab contains a simple reflected cross-site scripting vulnerability in the search functionality. User input is directly reflected back into the HTML response without any encoding or sanitisation, making it trivial to inject arbitrary HTML and JavaScript.

■ VULNERABILITY ANALYSIS

The search parameter value is reflected raw inside the HTML response body. The server performs no input validation or output encoding, so any HTML/JS tags submitted by the user are rendered by the browser.

■ EXPLOITATION STEPS

1. Navigate to the lab and locate the search box.
2. Type any text and observe how the input is reflected in the page source.
3. Open DevTools → Elements and confirm your input appears inside an HTML tag without encoding.
4. Enter the XSS payload into the search box and click Search.
5. The browser executes the script and an alert dialog appears — lab solved!

■ PAYLOAD(S)

■ Classic script-tag payload

```
<script>alert(1)</script>
```

■ WHY IT WORKS

Because angle brackets and script tags are not encoded, the injected script tag is parsed as valid HTML and executed immediately by the browser.

URL Slug	web-security/reflected/lab-html-context-nothing-encoded

LAB 02	STORED XSS	■ SOLVED	APPRENTICE

Stored XSS into HTML Context with Nothing Encoded

■ DESCRIPTION

This lab contains a stored cross-site scripting vulnerability in the blog comment functionality. The application stores user-supplied comment data and later renders it in the page without any output encoding, so the payload persists and fires for every visitor.

■ VULNERABILITY ANALYSIS

Comment text is stored in the database and later inserted directly into the HTML without escaping. Unlike reflected XSS, the payload executes for every user who views the blog post.

■ EXPLOITATION STEPS

1. Open any blog post and scroll to the comment section.
2. In the comment box, enter the XSS payload.
3. Fill in valid values for Name, Email, and Website fields.
4. Click "Post comment".
5. Return to the blog post. The alert fires as soon as the page loads.

■ PAYLOAD(S)

■ Stored in comment body

```
<script>alert(1)</script>
```

■ WHY IT WORKS

The comment content is written to the database and served back verbatim. Every subsequent page load triggers the injected script, affecting all visitors.

URL Slug	web-security/stored/lab-html-context-nothing-encoded

LAB 03	DOM-BASED XSS	■ SOLVED	APPRENTICE

DOM XSS in document.write Sink using location.search Source

■ DESCRIPTION

This lab contains a DOM-based cross-site scripting vulnerability in the search query tracking feature. The JavaScript uses document.write() to render data taken from location.search directly into the page, allowing an attacker to break out of the HTML context and inject executable code.

■ VULNERABILITY ANALYSIS

The client-side JavaScript reads window.location.search and passes the raw value to document.write(). The search term is placed inside an img src attribute, which can be escaped with a closing quote and angle bracket.

■ EXPLOITATION STEPS

1. Enter any random string in the search box and submit.
2. Open DevTools → Elements and inspect where your string appears in the DOM.
3. Observe that the string is placed inside an img src attribute.
4. Use the payload below to break out of the img attribute and inject an SVG element.
5. The onload handler fires and triggers the alert.

■ PAYLOAD(S)

■ Breaks out of img src attribute

```
"><svg onload=alert(1)>
```

■ WHY IT WORKS

The " closes the src attribute value, > closes the img tag, then the svg onload=alert(1) element is written directly into the DOM and executed.

URL Slug	web-security/dom-based/lab-document-write-sink

LAB 04	DOM-BASED XSS	■ SOLVED	APPRENTICE

DOM XSS in innerHTML Sink using location.search Source

■ DESCRIPTION

This lab contains a DOM-based cross-site scripting vulnerability in the search blog functionality. The application reads user input from location.search and assigns it to the innerHTML property of a div element, allowing injection of arbitrary HTML that executes JavaScript.

■ VULNERABILITY ANALYSIS

innerHTML does not execute script tags inserted at runtime, but it does execute event-handler attributes on other elements such as img. An attacker can use onerror to trigger JS execution when the image fails to load.

■ EXPLOITATION STEPS

1. Submit any text in the search box and inspect the page source.
2. Locate the div element whose innerHTML is set from the search query.
3. Note that script tags injected via innerHTML are not executed.
4. Use the img/onerror technique instead.
5. Click Search — the alert fires because src=1 is invalid and onerror executes.

■ PAYLOAD(S)

■ img onerror — works inside innerHTML

```
<img src=1 onerror=alert(1)>
```

■ WHY IT WORKS

Browsers deliberately prevent script tag execution via innerHTML. However, event handlers like onerror on img tags are still invoked, providing an alternative execution path.

URL Slug	web-security/dom-based/lab-innerhtml-sink

LAB 05	DOM-BASED XSS	■ SOLVED	APPRENTICE

DOM XSS in jQuery anchor href Attribute Sink using location.search Source

■ DESCRIPTION

This lab contains a DOM-based XSS vulnerability in the Submit Feedback page. jQuery's $ selector function is used to find an anchor element and update its href attribute using data from location.search, making it possible to inject a javascript: URI that executes when clicked.

■ VULNERABILITY ANALYSIS

The page reads the returnPath parameter from the URL and sets it as the href of the Back link using jQuery's .attr(). Because no validation is done, a javascript: URI can be injected, which the browser executes when the user clicks the link.

■ EXPLOITATION STEPS

1. Navigate to the Submit Feedback page.
2. Observe the returnPath parameter in the URL.
3. Change returnPath to a random string, right-click the Back link and inspect it.
4. Confirm your string appears as the href value.
5. Replace returnPath with the javascript: payload.
6. Click the "Back" link — the alert fires.

■ PAYLOAD(S)

■ Injected into href via returnPath param

```
javascript:alert(document.cookie)
```

■ WHY IT WORKS

jQuery's .attr('href', value) sets the href to whatever string is supplied. When the value is a javascript: URI, the browser executes the code on click, bypassing the need to inject HTML tags.

URL Slug	web-security/dom-based/lab-jquery-href-attribute-sink

LAB 06	DOM-BASED XSS	■ SOLVED	APPRENTICE

DOM XSS in jQuery Selector Sink using a hashchange Event

■ DESCRIPTION

This lab contains a DOM-based XSS vulnerability on the home page. The page uses jQuery's $() selector function on the location.hash property to auto-scroll to a given post. An attacker can abuse this by delivering a crafted URL with a malicious hash to an unsuspecting victim via the lab's exploit server.

■ VULNERABILITY ANALYSIS

jQuery's $() selector interprets strings starting with < as HTML and inserts them into the DOM. By setting location.hash to an img tag with an onerror handler, the attacker can execute arbitrary JavaScript in the victim's browser.

■ EXPLOITATION STEPS

1. Inspect the home page source and identify the hashchange event handler.
2. Open the exploit server from the lab banner.
3. Paste the iframe payload into the Body section (replace YOUR-LAB-ID).
4. Click "Store exploit", then "View exploit" to test — you should see print() fire.
5. Click "Deliver to victim" to solve the lab.

■ PAYLOAD(S)

■ Exploit server payload — triggers print() in victim's browser

```
<iframe src="https://YOUR-LAB-ID.web-security-academy.net/#" onload="this.src+='<img src=x onerror=print()>'"> </iframe>
```

■ WHY IT WORKS

The onload handler appends an img XSS payload to the iframe src as a hash fragment. jQuery's $() selector parses the hash as HTML and injects it into the DOM, triggering the onerror handler.

URL Slug	web-security/dom-based/lab-jquery-selector-hash-change-event

LAB 07	REFLECTED XSS	■ SOLVED	APPRENTICE

Reflected XSS into Attribute with Angle Brackets HTML-Encoded

■ DESCRIPTION

This lab contains a reflected XSS vulnerability in the search blog functionality. Angle brackets are HTML-encoded to prevent breaking out of tags, but the application does not prevent injection of new attributes. By injecting an event handler, arbitrary JavaScript can still be executed.

■ VULNERABILITY ANALYSIS

The search term is reflected inside a quoted HTML attribute. Angle brackets are encoded (< becomes <), but double-quotes are not escaped within the attribute value, allowing an attacker to close the attribute and inject a new event handler attribute.

■ EXPLOITATION STEPS

1. Submit a random alphanumeric string in the search box.
2. Use Burp Suite to intercept the request and send it to Burp Repeater.
3. Observe that the string appears inside a quoted HTML attribute.
4. Note that angle brackets are encoded but quotes are not.
5. Replace the search term with the payload below.
6. Copy the modified URL from Burp and open it in the browser. Hover over the injected element — alert fires.

■ PAYLOAD(S)

■ Injects event handler into quoted attribute

```
"onmouseover="alert(1)
```

■ WHY IT WORKS

The " closes the current attribute value, onmouseover="alert(1) adds a new event handler, and the browser executes alert(1) when the mouse moves over the element. Angle-bracket encoding is bypassed entirely.

URL Slug	web-security/contexts/lab-attribute-angle-brackets-html-encoded

LAB 08	STORED XSS	■ SOLVED	APPRENTICE

Stored XSS into anchor href Attribute with Double Quotes HTML-Encoded

■ DESCRIPTION

This lab contains a stored XSS vulnerability in the comment functionality. The Website field of the comment form is stored and later rendered as the href of an anchor tag. Double quotes are HTML-encoded, but a javascript: URI is not filtered, so the payload executes when the comment author name is clicked.

■ VULNERABILITY ANALYSIS

The Website field value is written into an anchor href attribute. Double quotes are encoded, preventing attribute injection, but javascript: URIs are not blocked. Clicking the author name triggers the payload.

■ EXPLOITATION STEPS

1. Navigate to a blog post and open the comment form.
2. In the Website field, enter a random alphanumeric string.
3. Use Burp Suite to intercept the POST request and send to Repeater.
4. Make a GET request to view the post and intercept it; send to Repeater.
5. Observe the random string reflected inside the anchor href.
6. Repeat the comment submission but replace the Website value with the payload.
7. Right-click → "Copy URL" from Burp and load it. Click the comment author name — alert fires.

■ PAYLOAD(S)

■ Injected into anchor href via Website field

```
javascript:alert(1)
```

■ WHY IT WORKS

Even with double-quote encoding, the javascript: scheme is allowed in href. When a user clicks the author name link, the browser executes the JavaScript in the URI, bypassing HTML-attribute encoding protections.

URL Slug	web-security/contexts/lab-href-attribute-double-quotes-html-encoded

LAB 09	REFLECTED XSS	■ SOLVED	APPRENTICE

Reflected XSS into a JavaScript String with Angle Brackets HTML-Encoded

■ DESCRIPTION

This lab contains a reflected XSS vulnerability in the search query tracking functionality. The search term is reflected inside a JavaScript string literal. Angle brackets are HTML-encoded, but single quotes are not, allowing an attacker to break out of the string and inject arbitrary JavaScript.

■ VULNERABILITY ANALYSIS

The server-side code places the search term inside a JS string: var searchTerm = 'USER_INPUT'; — angle brackets are encoded but single quotes are not, so the string can be terminated and arbitrary code appended.

■ EXPLOITATION STEPS

1. Submit a random alphanumeric string in the search box.
2. Use Burp Suite to intercept the search request and send to Repeater.
3. In the response, observe the string reflected inside a JS variable assignment.
4. Confirm angle brackets are encoded but single quotes are not.
5. Replace the search term with the payload.
6. Right-click → "Copy URL" and load it in the browser — alert fires on page load.

■ PAYLOAD(S)

■ Breaks out of JS string and calls alert

```
'-alert(1)-'
```

■ WHY IT WORKS

The first ' closes the string literal, -alert(1)- executes alert() as part of a subtraction expression (valid JS), and the trailing ' reopens a new string to prevent syntax errors — clean JS injection.

URL Slug	web-security/contexts/lab-javascript-string-angle-brackets-html-encoded

XSS Payload Quick-Reference Cheat Sheet

#	Context	Payload	Notes
01	Reflected XSS	`<script>alert(1)</script>`	Classic script-tag payload
02	Stored XSS	`<script>alert(1)</script>`	Stored in comment body
03	DOM-Based XSS	`"><svg onload=alert(1)>`	Breaks out of img src attribute
04	DOM-Based XSS	`<img src=1 onerror=alert(1)>`	img onerror — works inside innerHTML
05	DOM-Based XSS	`javascript:alert(document.cookie)`	Injected into href via returnPath param
06	DOM-Based XSS	`<iframe src="https://YOUR-LAB-ID.web-security-academy.net/#" onload="this.src+='<im </iframe>`	Exploit server payload — triggers print() in victim
07	Reflected XSS	`"onmouseover="alert(1)`	Injects event handler into quoted attribute
08	Stored XSS	`javascript:alert(1)`	Injected into anchor href via Website field
09	Reflected XSS	`'-alert(1)-'`	Breaks out of JS string and calls alert

Key Takeaway: XSS vulnerabilities arise when user-supplied data is included in a web page response without proper contextual output encoding. The correct defence depends on where data appears: HTML body → HTML-encode; HTML attribute → attribute-encode; JavaScript string → JS-encode; URL parameter → URL-encode. Always apply the right encoding for the right context.